

Paradigma: é um modelo, padrão ou estilo de programação suportado por linguagens que agrupam certas características comuns.

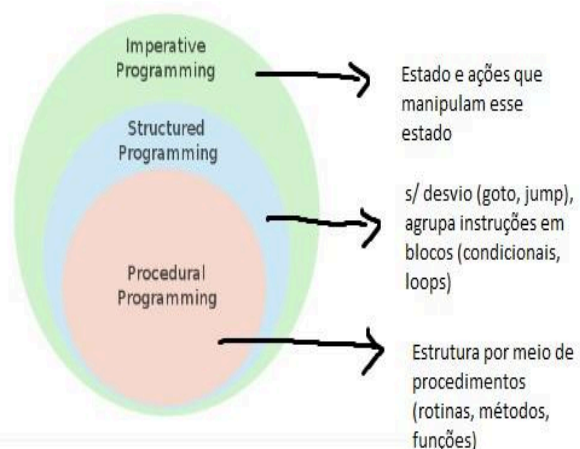
- ajudam a compreensão da implementação da solução
- provêm mecanismos para diferentes soluções

Paradigmas comuns:

- **Imperativo**: primeiro paradigma a ser definido e conceituado, segue o conceito de estado e ações que manipulam esse estado.
Faz o uso de variáveis que envolve (nome, endereço, valor, tipo, escopo, tempo de vida)
- **Estruturado**: Não utilizam instruções de desvio (goto, jump), agrupam instruções em blocos (condicionais, loops, indentação).
- **OO**: tudo é objeto, possui classes, objetos, métodos e atributos. Esconde o que não é importante.
- **Funcional**: trata a computação como avaliação de funções matemáticas, alto nível de abstração, estilo declarativo, sem expressões com efeitos colaterais.
- **Procedural**: Estruturam o código por meio de procedimentos (rotinas, métodos, funções)
- **Orientado a eventos**: aguarda um evento para realizar uma ação (cliques do mouse, teclado, etc)
- **Orientado a aspectos**: visa melhorar a OO através da separação de interesses transversais (logging, autenticação, controle de acesso, transações, concorrência, etc).
- **Orientado a fluxos**
- **Declarativo**: O programador define somente o que deseja, sem se preocupar em como obtê-lo. Quem executar/interpretar o código será responsável por obter os resultados.
- **Lógico**: Baseado em fatos e regras, inferência lógica de informações.

```
Structured:
IF x<=y THEN
  BEGIN
    z := y-x;
    q :=SQRT(z);
  END
ELSE
  BEGIN
    z := x-y;
    q := -SQRT(z)
  END;
WRITELN(z,q);
```

```
Unstructured:
IF x>y THEN GOTO 2;
z := y-x;
q := SQRT(z);
GOTO 1;
2: z:= x-y;
q:=-SQRT(z);
1: writeln(z,q);
```



Efeitos colaterais: ocorre quando a função modifica um de seus parâmetros ou uma variável global.

Paradigmas vs Linguagens

- Linguagens oferecem suporte para paradigmas particulares, também podendo suportar vários paradigmas, facilitando a programação.
- Poucas linguagens são puras (possuem suporte para apenas um paradigma exclusivamente).

PARADIGMA FUNCIONAL

ex. de linguagens: erlang, elixir, clojure, scala, ocaml, haskell, lisp

- Em funções matemáticas, a ordem de avaliação das expressões é controlada por recursão e condicionais. Enquanto na imperativa por sequência e repetição.
- Stateless: funções devem retornar o resultado como se fosse sua primeira chamada, tudo que rodou antes de sua chamada não deve influenciar no resultado.

Tem como objetivo mimetizar funções matemáticas ao máximo, enquanto no paradigma imperativo o resultado de uma expressão é guardado em um endereço de memória representada como uma variável, no paradigma funcional não existem variáveis, não possibilitando as construções de iteração.

- Não existem atribuições, existem correspondências.

PRINCÍPIOS DA PROGRAMAÇÃO FUNCIONAL

- **Funções puras:** agem sob os próprios parâmetros, devem sempre retornar algo, não possuem efeito colateral, sempre retornarão a mesma saída para a mesma entrada.
- **Imutabilidade:** uma vez que o valor é definido, não pode ser alterado. Facilitando a programação concorrente e apresentando códigos mais consistentes.
- **Funções de primeira ordem:** funções=dados, pode-se atribuí-las a variáveis, passar ou retornar por parâmetro.
- **Funções de alta ordem:** recebem funções como parâmetros e retornam funções.
- **Funções Lambda (anônima):** função que não está vinculada a um identificador/nome, é passada como argumento para outra função.
- **Composição:** Aplica uma função ao resultado de outra função, produzindo uma terceira função (novaFuncao = f(g(x))).
- **Currying:** funções que recebem um único parâmetro por vez, permitindo a aplicação parcial da função. O resultado será outra função.

FUNÇÕES COMUNS NA PROGRAMAÇÃO FUNCIONAL

Map: Aplica uma função, passada por parâmetro, em cada elemento em um arranjo e retorna o resultado.

Filter: Cria um novo arranjo com todos os elementos que satisfazem o teste implementado pela função passada por parâmetro.

Reduce: Aplica uma função sobre um acumulador e cada valor de um arranjo (da esquerda para a direita) para reduzir a um único valor.



CONCORRÊNCIA NA PROGRAMAÇÃO FUNCIONAL

A partir de funções puras podemos combinar a concorrência e criar códigos mais otimizados.

Concorrência: possibilidade de utilizar de um mesmo núcleo de forma concorrente, vários processos utilizando o mesmo núcleo.

Paralelismo: faz o uso de diferentes núcleos em paralelo em execução.

Funções síncronas: o sistema aguarda a execução para dar sequência ao funcionamento.

Funções assíncronas: é resolvida um tempo depois de sua execução, permitindo realizar outras atividades enquanto a função é executada. (ex.: `setInterval`, `setTimeout`, `async`, `await`, `Promises`).

Callbacks: chamadas de retorno, são funções invocadas após o fim de uma tarefa assíncrona, informando o término da tarefa.

Promises: forma de melhorar o conceito complexo de callback

Simplifica o uso de callback, tal que uma promise é uma referência que podemos consultar o estado da função. Retornando uma promessa do resultado.

- .then - Se foi executado com sucesso
- .catch - Se não foi executado com sucesso
- .finally - De qualquer forma faça isso

“A programação orientada a aspecto foi inventada para superar limites da organização hierárquica de classes”

Voltando à sua frase, "programação orientada a aspecto foi inventada pra superar limites da organização hierárquica de classes", ela significa que, enquanto a POO é excelente para organizar a lógica do seu negócio em hierarquias (como uma classe `Veiculo` e suas subclasses `Carro` e `Moto`), ela falha em lidar de forma limpa com funcionalidades que não respeitam essa hierarquia e precisam ser aplicadas em vários pontos do sistema.

A POA, ao introduzir o conceito de **aspecto**, oferece uma maneira de **modularizar** esses "concerns" transversais. Em vez de ter o código de log espalhado por 50 classes, você tem um único aspecto que "injeta" o comportamento de log onde e quando for necessário, de forma não intrusiva. Isso mantém o código da sua classe de negócio limpo e focado em sua responsabilidade principal, ao mesmo tempo que centraliza a lógica transversal.